İşletim Sistemleri
Process Senkronizasyonu 4

Dr. Öğr. Üyesi Ertan Bütün

Bu dersin içeriği hazırlanırken Operating System Concepts (Silberschatz, Galvin and Gagne) kitabı ve
Prof. Dr. M. Ali Akcayol'un (Gazi Üniversitesi Bilgisayar Mühendisliği Bölümü) ders sunumlarından faydalanılmıştır.

https://codex.cs.yale.edu/avi/os-book/OS9/slide-dir/index.html
http://w3.gazi.edu.tr/~akcayol/BMOS.htm

Problems with Semaphores

- Semaforlar, proses senkronizasyonu için uygun ve etkili bir mekanizma sağlasalar da, bunların yanlış kullanılması hatalara neden olabilir.
- Bu hatalar yalnızca belirli yürütme sırasında gerçekleştiğinde meydana geldiğinden ve bu yürütme sıraları her zaman ortaya çıkmadığından tespit edilmesi zor olan zamanlama hatalarına neden olabilir.
- Producer-consumer problemi çözümünde *count* değişkeninin belli yürütme sırasında (race condition) yanlış değer aldığını hatırlayalım. Bu örnekte, zamanlama hatası nadiren yalnızca belli yürütme sırası olduğunda meydana gelir.
- Tanıtılan mutex lock veya semafor kullanımında da bu tür zamanlama hataları meydana gelebilir.

Problems with Semaphores

- CS probleminin semafor çözümünü hatırlayalım: Tüm processler, 1 ile başlatılan binary bir semafor olan mutex değişkenini paylaşır.
- Aşağıda iki proses için verilen CS problemi çözümü düzgün çalışır mı?

```
while (true)
{
        wait(mutex)
            …
            critical section
            …
        signal(mutex)
}
```

```
while (true)
{
        signal(mutex)
            …
            critical section
            …
        wait(mutex)
}
```

Problems with Semaphores

- CS probleminin semafor çözümünü hatırlayalım: Tüm processler, 1 ile başlatılan binary bir semafor olan mutex değişkenini paylaşır.
  - Aşağıdaki senaryoda birden fazla proses aynı anda CS'de olabilir, mutual-exclusion ihlal edilebilir.

```
while (true)
{
        wait(mutex)
            …
            critical section
            …
        signal(mutex)
}
```

```
while (true)
{
        signal(mutex)
            …
            critical section
            …
        wait(mutex)
}
```

Problems with Semaphores

- CS probleminin semafor çözümünde her proses, CS'ye girmeden **önce** wait(mutex) ve CS'den **çıkarken** signal(mutex) yürütmelidir.
- Bu sıraya uyulmadığı takdirde iki proses aynı anda CS'de olabilir. Bu duruma bir programlama hatası veya işbirliği yapmayan bir programcı neden olabilir.

Problems with Semaphores

- Aşağıdaki CS çözümü düzgün çalışır mı?

```
while (true)
{
        wait(mutex)
            …
            critical section
            …
        wait(mutex)
}
```

Problems with Semaphores

- Bu durumda proses wait(mutex)'i ikinci çağırışında mutex'in değeri 0 olduğundan kalıcı olarak bloke olacaktır.

```
wait(mutex);
      ...
   critical section
      ...
wait(mutex);
```

- Ya da signal(mutex) ve wait(mutex) işlemlerinden biri veya her ikisi unutulduğunda ya mutual-exclusion ihlal edilecek, ya da proses kalıcı olarak bloke olacaktır.
- Bu örnekler, programcılar CS problemini çözmek için semaforları veya mutex kilitlerini yanlış kullandıklarında çeşitli hata türlerinin kolayca üretilebileceğini göstermektedir. Bu tür hatalarla başa çıkmak için bir strateji, basit senkronizasyon araçlarını yüksek seviye dillerdeki yapılara dahil etmektir.

Konular

- Process Synchronization
- The Critical-Section Problem
- Peterson’s Solution
- Synchronization Hardware
- Mutex Locks
- Semaphores
- Classical Problems of Synchronization
- Monitors
- Synchronization Examples
- Alternative Approaches

Monitors

- Semaforların kullanımıyla ilgili oluşabilecek hatalarla başa çıkmak için yüksek-seviye dil yapıları geliştirilmiştir, bunlardan biri de monitor'lerdir.
- Monitor proses senkronizasyonu için uygun ve etkili bir mekanizma sağlayan yüksek seviye bir soyutlamadır.

Monitors

- Monitor türünün genel yapısı şekilde verilmiştir.
- Soyut bir veri türü (abstract data type) olan **monitor** türü, mutual exclusion'ın sağlandığı programcıların tanımladığı bir dizi fonksiyonlardan (functions) oluşur.
- Monitör tipi ayrıca, değerleri bu tip bir monitör örneğin durumunu tanımlayan değişkenleri bildirir.
- Fonksiyonların içinde bu değişkenlerle ilgili işlemler yapılır.

```
monitor monitor name
{
   /* shared variable declarations */

   function P1 ( . . . ) {
      . . .
   }

   function P2 ( . . . ) {
      . . .
   }

         .
         .
         .
   function Pn ( . . . ) {
      . . .
   }

   initialization_code ( . . . ) {
      . . .
   }
}
```

Monitors

- Bir monitör tipi, çeşitli process'ler tarafından doğrudan kullanılamaz.
- Sadece, monitor içinde tanımlanan bir fonksiyon, monitor içinde tanımlanan lokal değişkenlere ve kendi parametrelerine erişebilir.
- **Monitor'de aynı anda yalnızca bir proses aktif olabilir.**
- **Monitor yapısı, monitörde aynı anda yalnızca bir process'in etkin olmasını garanti eder. Dolayısıyla, programcının senkronizasyon için bunu açıkça kodlaması gerekmez.**

Monitors

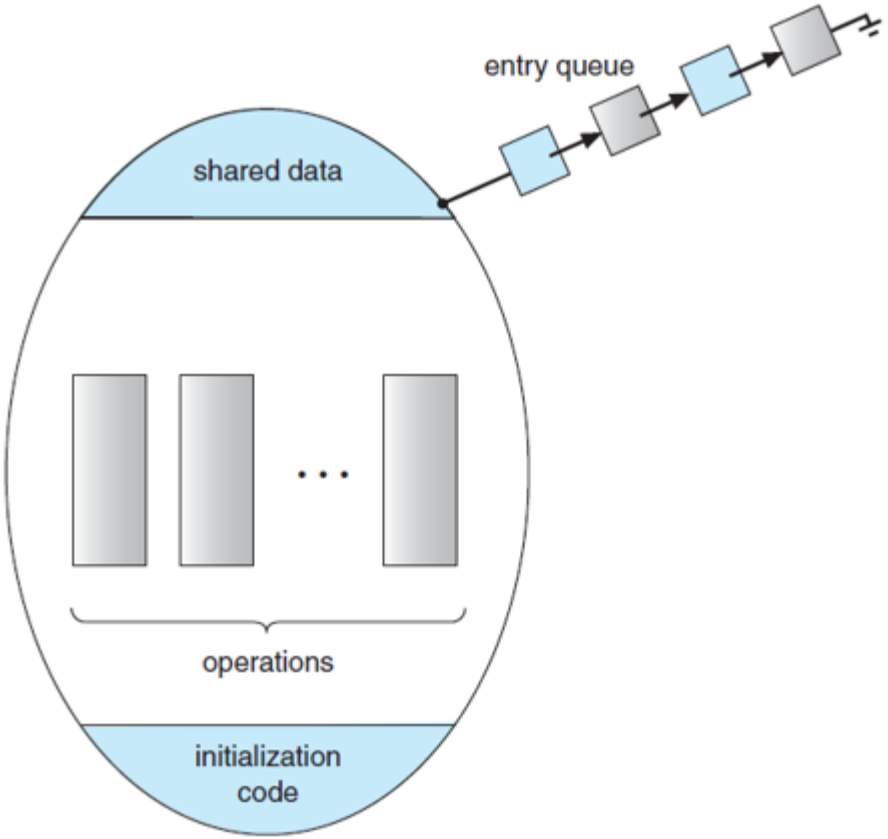
- Monitörler bir programlama dili yapısıdır, bu nedenle derleyici monitörlerin özel olduklarını bilir ve diğer fonksiyon çağrılarından farklı olarak monitör fonksiyonları için yapılan çağrıları onun çalışma mekanizmasına göre işler.
- Monitor girişlerinde nasıl bir yolla mutual exclusion uygulanacağı derleyiciye kalmıştır, ancak yaygın olan muteks veya binary semafor kullanmaktır.
- Karşılıklı dışlamayı programcı değil derleyici düzenlediğinden bir şeylerin ters gitme olasılığı çok daha düşüktür. Monitörü kullanan programcının, derleyicinin karşılıklı dışlamayı nasıl düzenlediğinin farkında olması gerekmez. Programcının tüm kritik bölgeleri monitor fonksiyonlarına çevirerek, hiçbir iki prosesin kendi kritik bölgelerini aynı anda yürütmeyeceğini bilmesi yeterlidir.

Monitors

- Tipik olarak, bir proses bir monitör fonksiyonunu çağırdığında fonksiyonun ilk birkaç komutu, monitörde o anda aktif olan başka bir proses olup olmadığını kontrol eder.
- Eğer aktif olan bir proses varsa, çağıran proses diğer proses monitörden ayrılana kadar askıya alınır. Monitörü kullanan başka bir proses yoksa, çağıran proses girebilir.

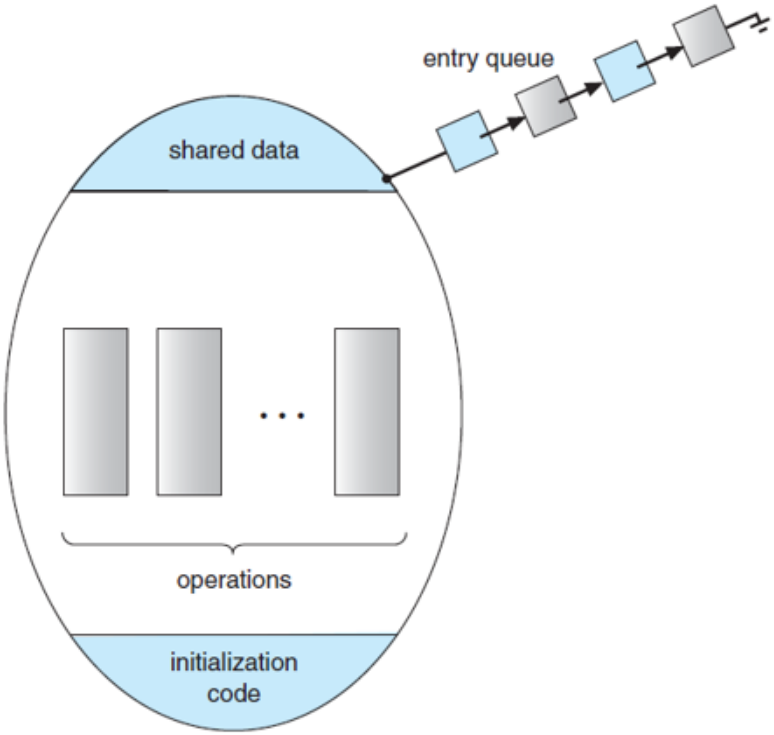
Monitors

- Bir monitor temelde kullanılan CS'yi **izler** yani monitor eder ismi de burdan geliyor zaten.
- Bir monitor içinde process'ler arasında paylaşılan veriler, ve bu veriler üzerinde işlemlerin gerçekleştirildiği fonksiyonlar vardır.



Monitors

- Bu paylaşılan veriler monitor içinde global olarak tanımlıdır, içindeki fonksiyonlar tarafından erişilir. Ayrıca bu paylaşılan verilere başlangıç değerlerinin verildiği bir kısım da vardır.
- Paylaşılan verilere ve bu fonksiyonlara erişmek için yani monitöre girmek için processler kuyruğa girer.



Monitors

- Monitörler mutual exclusion'ı sağlamanın kolay bir yolunu sağlasa da, bu yeterli değildir. Ayrıca, proseslerin monitörde senkronizasyon gereği devam edemediğinde yani beklemeleri gerektiğinde bloke edilmesi için bir yola ihtiyacımız var.
- Bu, üzerlerinde **wait()** ve **signal()** işlemlerinin çağrılabildiği **durum yapıları (condition construct)** tarafından sağlanır.
- Özel olarak hazırlanmış bir senkronizasyon şeması yazması gereken bir programcı, bir veya daha fazla **durum(condition) değişkenini** tanımlayabilir:
    - `condition x, y;`

Monitors

- Bir durum değişkeni üzerinden **sadece wait()** ve **signal()** işlemleri çağrılabilir:
- `x.wait();`

  Bir proses bir monitor fonksiyonunda devam edemeyeceğini anladığında ilgili durum değişkeninde wait() i çağırarak beklemeye geçer. Başka bir proses **x.signal()'i** çalıştırıncaya kadar bekleme durumundadır.
- `x.signal();`

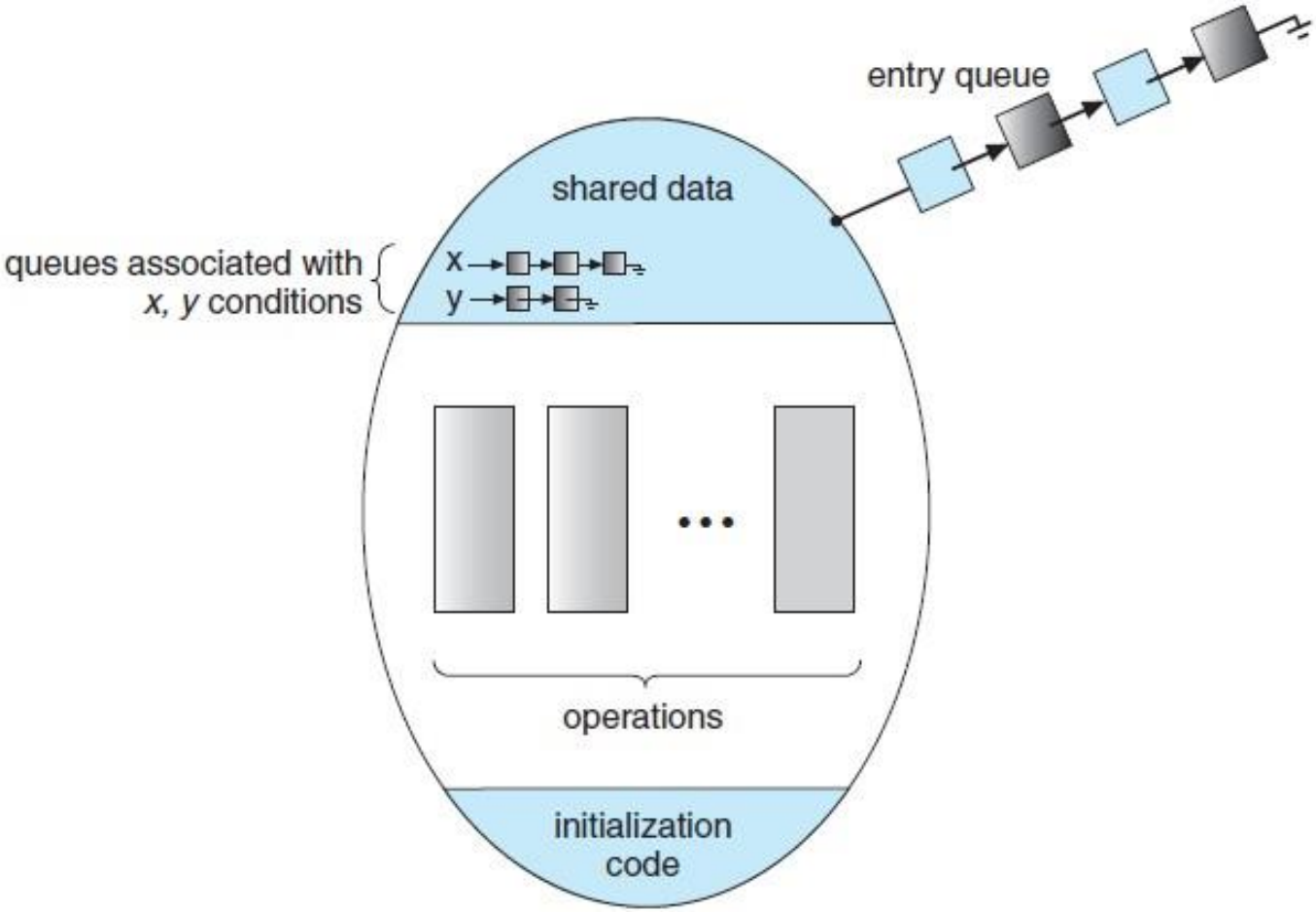
  ile **x condition** değişkenine bağlı beklemekte olan **tam olarak bir proses** çalışmaya devam eder.

  Hiçbir proses askıya alınmamışsa, **signal()** işleminin hiçbir etkisi olmaz; yani, x'in durumu, işlem hiç yürütülmemiş gibi aynıdır.

  Semafor kullanımında ise **signal()** işlemi her zaman semafor değişkeninin durumunu etkiler.

Monitors

- **x** ve **y** durum değişkenlerine bağlı process'lerin monitör içinde çalışmasının örnek gösterimi.



Condition Variable Kullanımına Örnek

- P_1 ve P_2 , S_1 ve S_2 komutlarını yürütmek isteyen processler olsun ve S_1'in S_2 'den önce yürütülmesine ihtiyaç olsun.
  - Sırasıyla P_1 ve P_2 tarafından çağrılan iki F_1 ve F_2 prosedürüyle bir monitör oluşturalım
  - Başlangıç değeri 0 olan bir "x" condition değişkeni ve "done" boolean değişkeni tanımlayalım
  - **F1:**

```
S1;
done = true;
x.signal();
```

  - **F2:**

```
if done = false
   x.wait()
S2;
```

Monitors

- **`x.signal()`** işlemini bir **`P`** process'i çağırmış olsun ve **`x`** durumuna bağlı beklemekte olan bir **`Q`** process'i olsun.
- Askıya alınan Q process'inin devam etmesine izin verilirse, signal veren P'nin beklemesi gerekir. Hem **`Q`** hem de **`P`** paralel olarak çalışamaz. **`Q`** devam ettirilirse, **`P`** beklemelidir.
- Q prosesi x durum değişkeni için beklemede iken P prosesi x.signal()i çağırdığında iki olasılık vardır:
  - Signal and wait: **`P`** process'i, **`Q`** process'inin monitör'den ayrılmasını veya **`Q'nun`** başka bir durum değişkeni için beklemeye geçmesini bekler.
  - Signal and continue: **`Q`** process'i, **`P`** process'inin monitör'den ayrılmasını veya **`P'nin`** başka bir durum değişkeni için beklemeye geçmesini bekler.

Monitors

- Yukarıdaki her iki seçeneğin de benimsenmesi lehine makul durumlar vardır. Bir yandan, P zaten monitörde yürütüldüğünden, **Signal and continue** yöntemi daha mantıklı görünür.
- Öte yandan, P'nin devam etmesine izin verirsek, yürütülme sırası daha sonra Q’ya geldiğinde, Q'nun beklediği durum artık geçerli olmayabilir.
- Bu iki seçenek arasında bir uzlaşma da mümkündür: P, signal() işlemini yürüttükten hemen sonra monitörden çıkar (monitörde beklemek yerine), Q hemen devam ettirilir. Yani, bir sinyal() çağırma işi bir monitör fonksiyonunun en sonunda yapılır böylece fonksiyondaki işler bitmiş olur ve ilgili proses monitörden çıkarılır.

Monitor Implementation Using Semaphores

- Signal and wait şemasına göre semafor kullanılarak monitör gerçekleştirimi :
- Değişkenler

```
semaphore mutex;      // (initially  = 1)
semaphore next;       // (initially  = 0)
int next_count = 0;   // monitor içinde
                      bekleyen process'lerin sayısı
```

- her bir **F** fonksiyonu şöyle değiştirilir:

```
wait(mutex);
      …
          body of F;
      …
if (next_count > 0)
      signal(next)
else
      signal(mutex);
```

- Böylece monitor içinde mutual exclusion sağlanır.

Monitor Implementation Using Semaphores

- Signal and wait şemasına göre semafor kullanılarak monitör gerçekleştirimi :
- Değişkenler - **Mutex**: mutual exclusion'ı sağlamak için her monitör için mutex isminde bir binary semafor sağlanır, 1 değeri ile başlatılır. Bir proses, monitöre girmeden önce wait (mutex) yürütmeli ve monitörden çıktıktan sonra signal (mutex) yürütmelidir
- Eğer bir proses monitöre girememişse, monitörün dışında wait(mutex) ile bekliyordur.
- her bir **F** fonksiyonu:

```
wait(mutex);
      …
         body of F;
      …
if (next_count > 0)
      signal(next)
else
      signal(mutex);
```

Monitor Implementation Using Semaphores

- Signal and wait şemasına göre semafor kullanılarak monitör gerçekleştirimi :
- Değişkenler - **Next** semaforu ise monitör içinde bekleyen proses için kullanılıyor.
- Burada verilen monitör gerçekleştirimi signal and wait şemasına göre verilmiştir. Bu şemaya göre signal işlemini gerçekleştirecek P prosesi, daha önce wait durumunda iken verilen signal ile kaldığı yerden devam ettirilecek Q prosesi işlerini bitirip ayrılana veya başka bir condition değişken için bekleme durumuna geçinceye kadar beklemesi gerektiğinden, **next** isimli binary semafor eklenir ve bu semafor 0 olarak başlatılır.
- Eğer bir proses monitör içinde ise **mutex** i almasına gerek yoktur zaten monitörün içindedir.

```
wait(mutex);
      …
        body of F;
      …
if (next_count > 0)
        signal(next)
else
        signal(mutex);
```

Monitor Implementation Using Semaphores

- Signal and wait şemasına göre semafor kullanılarak monitör gerçekleştirimi :
- Değişkenler - **next _count** değişkeni de, **next** ile monitör içinde beklemeye alınan proseslerin sayısını tutmak için kullanılır.
- Fonksiyon içinde yapılacak işlemler bittikten sonra fonksiyondan çıkarken öncelik, monitör içinde bekleyen processe verilir, **signal(next)** ile monitör içinde bekleyen bir sonraki process'e şans verilir.
- **next _count** sıfırsa yani monitör içinde bekleyen proses yoksa **mutex** serbest bırakılır, monitör dışında bekleyen bir sonraki process'e şans verilir.

```
wait(mutex);
      …
         body of F;
      …
if (next_count > 0)
      signal(next)
else
      signal(mutex);
```

Monitor Implementation Using Semaphores

Şimdi durum değişkenlerinin nasıl gerçekleştirildiğini de açıklayabiliriz
wait() gerçekleştirimi:

- Her **`x`** condition değişkeni için için **`x_sem`** semaforu ve **`x_count`** tamsayı değişkeni tanımlanır:

  `semaphore x_sem; //` **(initially = 0)**
  `int x_count = 0; //` **x condition değişkeni için bekleyenlerin sayısını tutuyor.**

- **`x.wait()`** işlemi şöyle gerçekleştirilebilir:

```
x_count++;
if (next_count > 0)
      signal(next);
else
      signal(mutex);
wait(x_sem);
x_count--;
```

Monitor Implementation Using Semaphores

wait() gerçekleştirimi:

- **x_count++** ile **x** condition için bekleyen process'lerin sayısı bir artırılıyor, çünkü **x** için burda **wait** işlemi yapılıyor
- Daha sonra monitör içinde bekleyen proses varsa sırada bekleyen bir sonraki proses **signal(next)** ile monitör içinde aktif olmaya başlar

```
x_count++;
if (next_count > 0)
      signal(next);
else
      signal(mutex);
wait(x_sem);
x_count--;
```

Monitor Implementation Using Semaphores

wait() gerçekleştirimi:

- Ardından **wait(x_sem)** çalıştırılıyor, zaten asıl yapılmak istenen buydu wait işlemiydi, **x.wait()** işlemini çağıran proses başka proses **x.signal()'**i çalıştırıncaya kadar bekleyecek.
- Başka proses **x.signal()'**i çalıştırınca **x_sem** için bekleyen söz konusu proses **x_sem**'i ele geçirecek bekleme durumundan çıkacak **x_count** değeri bir azaltılacak ardından paylaşılan veriler üzerinde yapmak istediği işleri yapacak.

```
x_count++;
if (next_count > 0)
      signal(next);
else
      signal(mutex);
wait(x_sem);
x_count--;
```

Monitor Implementation Using Semaphores

signal() gerçekleştirimi:

- Signal işleminde **x** durum değişkeni için bekleyen proses yoksa signal işlemi hiçbirşey yapmaz.
- Eğer varsa, **signal** işlemini gerçekleştiren proses signal and wait şemasına göre beklemeye alınacağı için monitör içinde bekleyen process'lerin sayısı yani **next_count** bir artırılıyor.

- **`x.signal()`** işlemi şöyle gerçekleştirilebilir:

```
if (x_count > 0) {
        next_count++;
        signal(x_sem);
        wait(next);
        next_count--;
}
```

Monitor Implementation Using Semaphores

signal() gerçekleştirimi:

- Daha sonra **signal(x_sem)** çalıştırılıyor, zaten **x.signal** ile asıl yapılmak istenen buydu, **x_sem** semaforuna signal uygulamaktı.

```
if (x_count > 0) {
      next_count++;
      signal(x_sem);
      wait(next);
      next_count--;
}
```

Monitor Implementation Using Semaphores

signal() gerçekleştirimi:

- Böylece **x_sem** semaforunu bekleyen başka bir proses devam edecek, ancak devam edebilmesi için signali çalıştıran söz konusu process'in signal and wait şemasına göre beklemeye geçmesi lazım o yüzden wait(next) çalıştırılıyor.
- Hem signali çalıştıran proses hem de signal ile uyandırlan proses aynı anda monitör içinde olamaz.

```
if (x_count > 0) {
      next_count++;
      signal(x_sem);
      wait(next);
      next_count--;
}
```

Resuming Processes within a Monitor

- **x** condition için birkaç proses askıya alınmışsa ve bir proses tarafından **x.signal ()** işlemi yürütüldüğünde askıya alınan işlemlerden hangisi daha sonra devam ettirilecek?
- Basit bir çözüm, ilk gelene ilk hizmet (FCFS) sıralaması kullanmaktır, böylece en uzun süredir bekleyen proses önce devam ettirilir.
- Ancak birçok durumda, bu kadar basit bir programlama şeması yeterli değildir. Bu amaçla, conditional-wait yapısı kullanılabilir:
  - `x.wait(c);`
  - `c`, priority number olarak adlandırılan öncelik numarasıdır.
  - En küçük öncelik numarasına (yüksek öncelik) sahip proses, bir sonraki adımda devam ettirilir.

Monitor Example Using Priority Number - Single Resource allocation

- conditional-wait yapısı kullanılarak rekabet içindeki processler arasında tek bir kaynağın tahsisini kontrolü eden bir resource allocator monitörü örneği verilmiştir.
- Söz konusu kaynağa erişmesi gereken bir proses aşağıdaki sırayı izlemelidir:

  R.acquire(t);
  …
  kaynağa erişim
  …
  R.release();
  //R, ResourceAllacator türünün bir örneğidir

```
monitor ResourceAllocator
{
   boolean busy;
   condition x;

   void acquire(int time) {
      if (busy)
         x.wait(time);
      busy = true;
   }

   void release() {
      busy = false;
      x.signal();
   }

   initialization_code() {
      busy = false;
   }
}
```

Monitor Example Using Priority Number - Single Resource allocation

- Her proses, bu kaynağın tahsisini talep ederken, kaynağı kullanmayı planladığı maksimum süreyi belirtir. Monitor, kaynağı en kısa süreliğine talep eden processe tahsis eder.
- Acquire metodunda kaynak kullanımda ise yani busy true ise söz konusu proses wait ile bekler, değilse kaynağı alır ve busy true olarak set eder, çünkü artık kaynak kendisindedir.
- Kaynakla işi bittikten sonra release fonksiyonu ile kaynağı bırakır, kaynak için bekleyen processe signal gönderir.

```
monitor ResourceAllocator
{
   boolean busy;
   condition x;

   void acquire(int time) {
      if (busy)
         x.wait(time);
      busy = true;
   }

   void release() {
      busy = false;
      x.signal();
   }

   initialization_code() {
      busy = false;
   }
}
```

Monitor Solution to Dining Philosophers

```
monitor DiningPhilosophers
{
   enum {THINKING, HUNGRY, EATING} state[5];
   condition self[5];

   void pickup(int i) {
      state[i] = HUNGRY;
      test(i);
      if (state[i] != EATING)
         self[i].wait();
   }

   void putdown(int i) {
      state[i] = THINKING;
      test((i + 4) % 5);
      test((i + 1) % 5);
   }
```

```
   void test(int i) {
      if ((state[(i + 4) % 5] != EATING) &&
       (state[i] == HUNGRY) &&
       (state[(i + 1) % 5] != EATING)) {
         state[i] = EATING;
         self[i].signal();
      }
   }

   initialization_code() {
      for (int i = 0; i < 5; i++)
         state[i] = THINKING;
   }
}
```

```
DiningPhilosophers.pickup(i);
                ...
                eat
                ...
DiningPhilosophers.putdown(i);
```

Konular

- Process Synchronization
- The Critical-Section Problem
- Peterson's Solution
- Synchronization Hardware
- Mutex Locks
- Semaphores
- Classical Problems of Synchronization
- Monitors
- Synchronization Examples
- Alternative Approaches

Kernel Synchronization - Windows

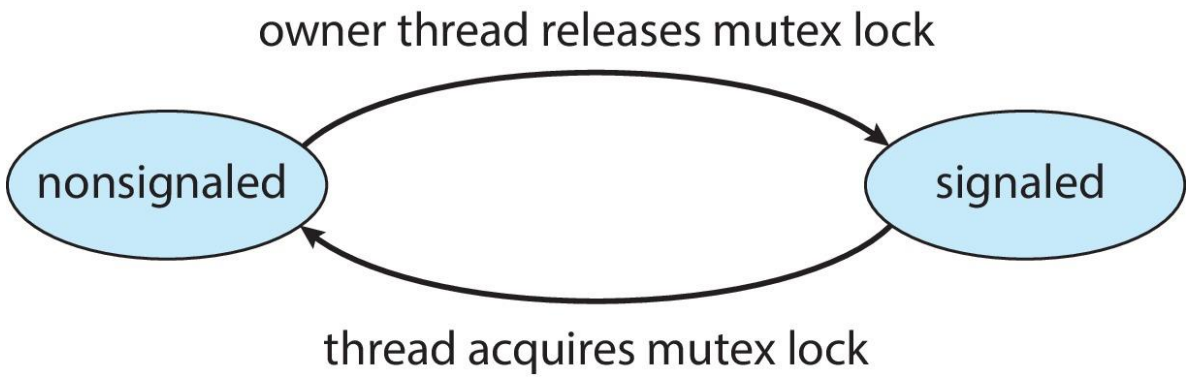
- Windows kernel tek işlemcili bir sistemde global bir kaynağa eriştiğinde, global kaynağa erişebilecek tüm kesme işleyicileri için kesmeleri geçici olarak maskeler.
- Çok işlemcili bir sistemde Windows, spinlocks kullanarak global kaynaklara erişimi korur.
  - spinlocks yalnızca kısa kod parçalarını korumak için kullanılır
  - Ayrıca, verimlilik için kernel, bir thread'in bir spinlock'ı tutarken asla preempted olmamasını sağlar.

Kernel Synchronization - Windows

- Windows kernel'in dışında thread senkronizasyonu için **dispatcher objects** sağlar.
- Bir **dispatcher object** kullanarak thread'ler; mutex locks, semaforlar, events ve timers dahil olmak üzere birkaç farklı mekanizmaya göre senkronize edilir.
  - **Events** condition değişkenlerine benzer; yani, istenen bir durum oluştuğunda bekleyen bir thread'e haber verebilirler.
  - **Timers** Timers, belirli bir sürenin dolduğunu bir (veya birden fazla) thread'e bildirmek için kullanılır.

Kernel Synchronization - Windows

- **Dispatcher objects**, **signaled** durumda veya **unsignaled** durumda olabilir.
  - **signaled** durumdaki bir obje müsaittir ve bir thread nesneyi alırken bloke olmaz.
  - **unsignaled** durumdaki bir nesne müsait değildir ve nesneyi almaya çalışırken bir thread engellenecektir.
- Şekilde mutex **dispatcher object** verilmiştir.



Linux Synchronization

- Linux kernel'de basit matematiksel işlemlerin senkronizasyonu için basit bir teknik olan atomic değişkenler kullanılır

  `atomic_t` bir atomic integer türüdür.

- Örneğin

```
atomic_t counter;
int value;
```

Atomic Operation	*Effect*
`atomic_set(&counter,5);`	`counter = 5`
`atomic_add(10,&counter);`	`counter = counter + 10`
`atomic_sub(4,&counter);`	`counter = counter - 4`
`atomic_inc(&counter);`	`counter = counter + 1`
`value = atomic_read(&counter);`	`value = 12`

- Counter değişkeni üzerinde yapılan atomic işlemler interrupt'a müsaade edilmeden gerçekleşir,
  - atomic işlemler yürütülürken başka bir processin interrupt'la atomic değişken üzerinde değişiklik yapmasına izin verilmez.

Linux Synchronization

- Linux:
  - Prior to kernel Version 2.6, disables interrupts to implement short critical sections
  - Version 2.6 and later, fully preemptive
- Linux provides:
  - Semaphores
  - atomic integers
  - spinlocks
  - reader-writer versions of both (spinlocks and semaphores)
- On single-cpu system, spinlocks replaced by enabling and disabling kernel preemption

Pthreads Synchronization

- Pthreads API is OS-independent
- It provides:
  - mutex locks
  - condition variable
- Non-portable extensions include:
  - read-write locks
  - spinlocks

Konular

- Process Synchronization
- The Critical-Section Problem
- Peterson’s Solution
- Synchronization Hardware
- Mutex Locks
- Semaphores
- Classical Problems of Synchronization
- Monitors
- Synchronization Examples
- Alternative Approaches

Alternative Approaches

Transactional memory

- **Multicore sistemlerde**, **mutex lock**, **semafor** gibi mekanizmalarda **deadlock** gibi problemlerin **oluşma riski bulunmaktadır.**
- Bunun yanı sıra, **thread sayısı arttıkça deadlock problemlerinin ortaya çıkma olasılığı artmaktadır.**
- **Klasik mutex lock (veya semafor)** kullanılarak paylaşılmış veride güncelleme yapan **update()** fonksiyonu aşağıdaki gibi yazılabilir.

```
void update ()
{
   acquire();

   /* modify shared data */

   release();
}
```

Alternative Approaches

Transactional memory

- Klasik kilitleme yöntemlerine alternatif olarak programlama dillerine yeni özellikler eklenmiştir.
- Örneğin, **atomic(S)** kullanılarak **S** işlemlerinin tümünün transaction olarak gerçekleştirilmesi sağlanır.

```
void update ()
{
   atomic {
      /* modify shared data */
   }
}
```

- Lock kullanımı yerine böyle bir mekanizma kullanmanın avantajı, atomikliği garanti etmekten geliştiricinin değil transacitonal memory sisteminin sorumlu olmasıdır.
- Ayrıca herhangi bir lock kullanımı söz konusu olmadığından deadlock mümkün değildir.

Alternative Approaches

OpenMP (Open Multi-Processing)

- OpenMP, C, C++ ve Fortran için compiler direktiflerinden oluşan API'dir.
- OpenMP, paylaşılmış hafızada eşzamanlı çalışmayı destekler.
- OpenMP, **`#pragma omp critical`** komutu ile critical sectionı belirler ve aynı anda sadece bir thread çalışmasına izin verir.

```
void update(int value)
{
    #pragma omp critical
    {
        counter += value;
    }
}
```